BAEKSEOK UNIVERSITY

ch03 Variables

목차

01 Variables

Variables

Variables

- Variables are containers for storing data values, like numbers and characters.
- You can think of a variable as a named box where you keep a value that can be used later.
- In C, variables must have a specific type, which tells the program what kind of data the variable can store.
  - int - stores whole numbers (integers), such as 123 or -123
  - float - stores numbers with decimals, such as 19.99 or -19.99
  - char - stores a single character, such as 'a' or 'B'. Characters are surrounded by single quotes

Declaring (Creating) Variables

- To create a variable, you must specify the type and give the variable a name.
- You can also assign a value at the same time:

Variables

- Syntax

```
type variableName = value;
```

- Where type is one of the C data types (such as int), and variableName is the name you choose (for example x or myNum).
- The equal sign = is used to assign a value to the variable.
- So, to create a variable that should store a number, look at the following example:

C Format Specifiers

Format Specifiers

- Format specifiers are used together with the printf() function to print variables.
- You can think of a format specifier as a placeholder that tells C what kind of value will be printed.
- A format specifier always starts with a percentage sign %, followed by a letter.
- For example, to output the value of an int variable, use the format specifier %d surrounded by double quotes (""), inside the printf() function:

```
int myNum = 15;
printf("%d", myNum); // Outputs 15
```

C Format Specifiers

- Different data types use different format specifiers:

```c
// Create variables
int myNum = 15;             // Integer (whole number)
float myFloatNum = 5.99;    // Floating point number
char myLetter = 'D';        // Character

// Print variables
printf("%d\n", myNum);
printf("%f\n", myFloatNum);
printf("%c\n", myLetter)
```

- To combine both text and a variable, separate them with a comma inside the printf() function:

```c
int myNum = 15;
printf("My favorite number is: %d", myNum);
```

naming and declaration rules

- C Variable Names
  - All C variables must be identified with unique names.
  - These unique names are called identifiers.
  - Identifiers can be short names (like x and y) or more descriptive names (age, sum, totalVolume).

- Note: It is recommended to use descriptive names in order to create understandable and maintainable code:

naming and declaration rules

1. Must start with a letter or underscore

- A variable name cannot start with a number.
- It must start with:
  - a letter (A-Z or a-z)
  - or an underscore (_)

```
int age;
int _count;
int totalMarks;
```

```
int 2age;
int #count;
```

2. Can contain letters, numbers, and underscore

After the first character, you can use:

- letters
- numbers
- underscore _

```
int student1;
int total_marks;
```

naming and declaration rules

3. No spaces allowed

```
int total marks;
```

4. Cannot use C keywords

```
int int;
int return;
```

5. Case sensitive

```
int age;
int Age;
```

6. Must be declared before use

You must **declare the variable before using it.**

```
#include <stdio.h>

int main() {
    int age;   // declaration
    age = 20;  // assignment
    printf("%d", age);
    return 0;
}
```

Change Variable Values

- If you assign a new value to an existing variable, the new value will replace the old one:

```
int myNum = 15;  // myNum is 15
myNum = 10;  // Now myNum is 10
```

- You can also assign the value of one variable to another:

```
int myNum = 15;

int myOtherNum = 23;

// Assign the value of myOtherNum (23) to myNum
myNum = myOtherNum;

// myNum is now 23 (was 15)
printf("%d", myNum);
```

Declare Multiple Variables

Declare Multiple Variables

- To declare more than one variable of the same type, use a comma-separated list:

```
int x = 5, y = 6, z = 50;
printf("%d", x + y + z);
```

- You can also assign the **same value** to multiple variables at once:

```
int x, y, z;
x = y = z = 50;
printf("%d", x + y + z);
```

Example

- Real-Life Example
  - Often in our examples, we simplify variable names to match their data type (myInt or myNum for int types, myChar for char types, and so on). This is done to avoid confusion.
  - In real programs, variables often represent real-world data.
  - The following example stores information about a college student:

```c
#include <stdio.h>

int main() {
  // Student data
  int studentID = 15;
  int studentAge = 23;
  float studentFee = 75.25;
  char studentGrade = 'B';

  // Print variables
  printf("Student id: %d\n", studentID);
  printf("Student age: %d\n", studentAge);
  printf("Student fee: %f\n", studentFee);
  printf("Student grade: %c", studentGrade);

  return 0;
}
```

Code Challenge

- Calculate the Area of a Rectangle
- Instructions
  - Inside main(), complete the following steps:

1. Declare int variables named length and width, and assign them values
2. Declare int area and calculate it using length * width
3. Print the value of area using printf and use the correct format specifier

- Note: Remember to end a statement with semicolon.
- set on length = 2, width = 10

```c
#include <stdio.h>

int main() {
  // Write length here
  // Write width here
  // Write area here (length * width)

  // Print area here
  return 0;
}

```

BAEKSEOK UNIVERSITY

Thank You !

Copyright 2026
All rights reserved.